

YOUR LEARNING PLAN

AI ENGINEERING TRACK

Personalized Skill Development Plan — RockAI Dev

Intern Name:	Sohayl Islam El-Mahdy
Specialty Track:	AI Engineering
Plan Edition:	v2.0 (Detailed Execution)
Audit Scope:	Direct-address Personalized Learning Schedule

Your Learning Plan (Detailed Execution Edition)

Track: AI Engineering · Cycle length: 2 weeks · Prepared by: TPM/Training

How This Plan Is Budgeted (read this first)

Parameter	Value
Work week	Sunday–Thursday (5 learning days/week)
Daily learning block	1–2 hours (planned at 1.5h/day average)
Weekly budget	~7.5 hours
Cycle budget (2 weeks)	~15 hours
Planning rule	Core content is capped at ~12h/cycle (80%) — the remaining ~3h is buffer for overrun, re-reading, and blockers. Same 80% capacity rule we use for sprints, applied to learning.
Home learning	Optional acceleration only. The plan must be completable inside work hours alone.
Slippage rule	If a cycle slips, extend it — never compress the deliverable or skip the checkpoint . A skipped checkpoint is a fake checkpoint.
Logging	Every learning block is logged in Orbit as a <code>learning</code> task (base 6 pts) so time investment is visible next to project work.

Every source below is **scoped** (which sections/videos, estimated hours). Totals per cycle stay inside budget. If you finish early, stretch items are marked ★ — they are optional, not silently expected.

Personalization principle for this plan: you are the opposite risk profile from a typical junior AI hire — you've built a neural network's forward pass, backprop, and Adam optimizer entirely from scratch in raw NumPy, and you have a real, evidence-backed performance-debugging story (vectorizing a training loop for a ~40–60x speedup). The gap isn't ML depth, it's that **everything you've built is training/research-shaped, and RockAI's actual AI work is applied — RAG pipelines and LLM APIs wired into real products**. you said this himself: "haven't done much research in the RAG part." So this plan treats your from-scratch instinct as the asset to lean on (e.g. understanding what LangChain does *before* trusting it as a black box) while building the one skill category — LLM API integration and applied RAG — that your CV genuinely doesn't show yet.

Scope note, said to you directly and kindly: your Qiskit certifications and RL-for-games work are genuinely impressive and not being dismissed — they're just not what RockAI's applied-AI product focus needs from these 1–2 daily hours right now. You already told the interviewer you knows RAG is your gap; this plan takes you at your word and spends the budget there.

Cycle 1 (Weeks 1–2): LLM API Integration Fundamentals — closes G-02 ●

Budget: 14h (7h/week) · Prerequisite: none · No Arabic content exists for this topic — flagged, not a gap in the research.

Competency outcomes — by the end of this cycle you can:

1. Make a real LLM API call (Anthropic or OpenAI SDK) with proper request/response handling — not a copy-pasted quickstart snippet you doesn't fully understand.
2. Handle streaming responses, and explain when streaming matters for UX vs. when a blocking call is fine.
3. Implement retry/backoff for rate limits and transient errors — the difference between a demo and something that survives production traffic.
4. Parse structured output from a model reliably (JSON mode / tool-use style output) rather than regex-scraping free text.
5. Reason about basic cost/model-routing tradeoffs (which model tier for which task) — a decision you's never had to make when training your own models from scratch.

Week 1 — Foundation (7h)

Day	Time	Content	Source (scoped)
Sun	1.5h	Read with one question in mind: "what does calling someone else's trained model require that training my own never did?" First real API call, happy path only.	LLM API Integration — First Call to Production (2026) — intro + first-call sections (~45 min) + hands-on
Mon	1.5h	Anthropic SDK specifics: message format, system prompts, token counting. Flagged in Open Questions: provider choice is unconfirmed — use Anthropic as the default reference since it's the most likely fit, but keep the OpenAI SDK docs open as a fallback.	Anthropic API Documentation — Getting Started + Messages sections
Tue	1.5h	Streaming responses: implement a streamed call, compare the UX difference against Monday's blocking call side by side.	Docs — Streaming section + hands-on
Wed	1.5h	Error handling: rate limits, timeouts, malformed responses. Implement retry with exponential backoff —	LLM API Integration guide — error handling/retry section

Day	Time	Content	Source (scoped)
		treat this like the vectorization story: research why naive retry is wrong before writing it.	
Thu	1h	Structured output: request JSON-formatted responses and parse them reliably. Sketch which of RockAI's actual products (Amlak/Okade property data, Orbit tasks) this pattern would slot into.	Docs — structured output / tool-use section + design sketch

Week 2 — Real Feature (7h)

Day	Time	Content	Source (scoped)
Sun	1.5h	Pick one real, product-tied use case (e.g. auto-summarizing a property listing description, or drafting a task description for Orbit) and scope it in writing before coding.	Design note
Mon	1.5h	Build the happy path end-to-end.	Deliverable work
Tue	1.5h	Add error handling and retry/backoff to the real feature (not just the toy exercise from Wednesday).	Deliverable work
Wed	1.5h	Add structured output parsing + a basic cost/model-choice justification note (one paragraph: which model tier, why).	Deliverable work
Thu	1h	Self-review against the checkpoint list below.	Prep

Deliverable (specified, not vague)

One small, real feature — not a toy chatbot demo — that calls an LLM API to do something useful tied to an actual RockAI product domain (e.g. summarizing a property listing, drafting a task description). Must include: proper error handling, retry/backoff on rate limits, structured

output parsing, and a one-paragraph note justifying the model/cost choice. Pushed as a PR to your learning repo.

Checkpoint — pass criteria (mentor review)

- [] The feature survives a simulated rate-limit error without crashing (mentor forces one live)
- [] Structured output is parsed properly — no regex-scraping free text
- [] you can explain, unaided, why naive immediate-retry is worse than backoff
- [] you can articulate the cost/model-tier tradeoff for this specific feature, not just recite that "smaller models are cheaper"

★ Stretch: add basic token-usage logging so cost is visible per call, not just estimated.

Cycle 2 (Weeks 3–4): Embeddings & Vector Databases, Applied — closes G-04 🌀

Budget: 13h · Prerequisite: Cycle 1 deliverable merged

Competency outcomes:

6. Explain what an embedding actually is beyond your current high-level answer ("convert tokens to vectors") — dimensionality, semantic distance, and why cosine similarity is the typical comparison metric.
7. Build a chunk → embed → store pipeline end-to-end.
8. Query a vector store and evaluate whether the returned results are actually semantically relevant — not just "it ran without erroring."
9. Explain chunking strategy tradeoffs (chunk size, overlap) and why bad chunking silently produces bad retrieval later.

Week 1 — Concepts & First Pipeline (6.5h)

Day	Time	Content	Source (scoped)
Sun	1.5h	Embeddings deep dive: what the vectors actually encode, semantic distance, cosine similarity. Write your own corrected explanation vs. your interview answer — a direct, measurable before/after.	LangChain Docs — Semantic Search / Vector Stores — embeddings section (~45 min)
Mon	1.5h	Chunking strategies: fixed-size vs. semantic chunking, overlap, why chunk boundaries matter for retrieval quality.	Docs — chunking section + hands-on with a sample document
Tue	1.5h	Data ingestion → vector DB pipeline, built hands-on alongside the video (scoped to ingestion-	Build RAG Pipeline From Scratch (Part 1) — Krish Naik — full video, scoped to

Day	Time	Content	Source (scoped)
		through-storage only, not generation — that's Cycle 3).	ingestion/vector-DB content
Wed	1.5h	Querying the vector store: run test queries, inspect what actually comes back, and judge relevance manually before trusting any automated metric.	Docs + hands-on
Thu	0.5h	Pick the real/realistic dataset for the deliverable (RockAI internal docs or sample property listings) with mentor input if needed.	Design note

Week 2 — Applied Semantic Search (6.5h)

Day	Time	Content
Sun–Mon	3h	Build the full pipeline on the chosen dataset: chunk → embed → store.
Tue	1.5h	Build the query/retrieval layer + a small set of test queries with manually-judged expected results.
Wed	1h	Compare a couple of chunking configurations against the same test queries — does chunk size actually change what comes back?
Thu	0.5h	Checkpoint prep.

Deliverable

A small semantic search feature over a real or realistic dataset (RockAI internal docs or sample property listings): chunking pipeline, embedding + storage, a query interface, and a written note comparing at least two chunking configurations against a fixed set of test queries.

Checkpoint — pass criteria (mentor evaluates retrieval quality, not just "it runs")

- [] For a set of test queries the mentor picks live, the top results are actually relevant — judged by reading them, not by a green checkmark
- [] you can explain why one chunking configuration outperformed another on a specific query, with the actual result as evidence
- [] your corrected embeddings explanation (vs. your interview answer) is precise, not just longer

Cycle 3 (Weeks 5–6): Full RAG Pipeline — closes G-01, the company's stated top priority ●

Budget: 14h · Prerequisite: Cycle 2 pipeline · This is your single highest-priority cycle in the entire plan.

Competency outcomes:

1. Build a complete retrieval→generation loop: chunk → embed → store → retrieve → generate, wired together end to end.
2. Explain where hallucination can creep in at each stage of the pipeline (bad chunking, weak retrieval, or the generation step ignoring retrieved context).
3. Decide, with a real justification, whether to treat a RAG framework (LangChain) as a trusted black box or to reimplement a piece manually — leaning on your from-scratch instinct where it actually adds understanding, not everywhere out of habit.
4. Run a basic faithfulness check by hand: does the generated answer actually match what's in the source documents?

Week 1 — Full Pipeline Build (7h)

Day	Time	Content	Source (scoped)
Sun	1.5h	Read the retrieval→generation architecture with your Cycle 2 pipeline already in hand — this cycle extends what you built, it doesn't restart it.	LangChain Docs — Build a RAG Agent — generation/agent section
Mon	1.5h	Begin the freeCodeCamp course, scoped to indexing + retrieval review (fast pass, since Cycle 2 already covered this ground) then into the generation chapters.	Production RAG with LangChain & Vector Databases — freeCodeCamp — indexing review + generation chapters
Tue	1.5h	Wire retrieval output into the generation prompt: context injection, prompt structure for grounding the model in retrieved chunks.	Course continuation + hands-on
Wed	1.5h	From-scratch checkpoint, playing to your strongest instinct: pick one piece of the pipeline (e.g. the similarity-	Course reference + hands-on manual implementation

Day	Time	Content	Source (scoped)
		scoring/retrieval-ranking step) and reimplement it manually instead of using LangChain's black-box version — proven approach, same instinct as your NN-from-scratch project.	
Thu	1h	Continue the freeCodeCamp course into observability/basic agentic patterns — enough to know they exist, not mastery yet (mastery isn't this cycle's job).	Course — observability chapter (scoped)

Week 2 — Real Deliverable + Faithfulness (7h)

Day	Time	Content
Sun–Mon	4h	Build the full end-to-end RAG system on a real document set tied to an actual RockAI use case, wiring Cycle 2's retrieval into a generation step.
Tue	1.5h	Write 5–10 real test questions with manually-verified correct answers from the source documents — your own ground truth, not borrowed from a tutorial.
Wed	1h	Run the faithfulness check by hand: for each answer, does it actually match the source, or does it hallucinate anything not present?
Thu	0.5h	Checkpoint prep.

Deliverable

A working end-to-end RAG system (chunk → embed → store → retrieve → generate) answering real questions over a real document set tied to an actual RockAI use case, plus a hand-verified faithfulness check against 5–10 test questions with known-correct answers.

Checkpoint — pass criteria (mentor review, including a faithfulness spot-check)

- [] Mentor picks 2–3 of your test questions live and judges the answers against the source documents personally — not just trusting your self-report
- [] you can point to a specific pipeline stage and explain how a hallucination could originate there
- [] The manually-reimplemented piece (Wednesday, Week 1) actually works and you can explain what LangChain does under the hood at that point, unaided
- [] **Named win:** this closes your own self-identified top gap — worth stating plainly at the checkpoint, not just moving on to Cycle 4

Cycle 4 (Weeks 7–8): LLM Application Evaluation — closes G-03 🎯

Budget: 12h · Prerequisite: Cycle 3 RAG system

Competency outcomes:

5. Explain what a RAG-specific evaluation framework measures (faithfulness, answer relevancy, context precision/recall) and why generic ML metrics don't fit this problem well.
6. Add a working evaluation harness to your own Cycle 3 system, not a separate toy example.
7. Distinguish LLM-as-judge evaluation from embedding-based hallucination detection, and know when each is appropriate.
8. Measure and report latency alongside quality — evaluation isn't just "is the answer good," it's also "is it fast enough to ship."

Week 1 — Evaluation Concepts (6h)

Day	Time	Content	Source (scoped)
Sun	1.5h	RAGAS core metrics: faithfulness, answer relevancy, context precision/recall — read against your own Cycle 3 outputs as live examples, not abstract definitions.	Ragas Docs — Evaluation (~50 min)
Mon	1.5h	Hallucination detection approaches: LLM-as-judge vs. embedding-based detection — when each is worth the extra cost/latency.	Hallucination Detection and Evaluation in RAG Systems — RAGAS and Lynx (13 min, focused)
Tue	1.5h	Set up the RAGAS harness against your own Cycle 3 test question set from last cycle — reusing your own ground truth rather than a new dataset.	Docs + hands-on
Wed	1h	Add latency measurement per	Hands-on

Day	Time	Content	Source (scoped)
		query (retrieval time + generation time separately, not just total).	
Thu	0.5h	Review the first evaluation run's numbers — do they match your own manual faithfulness judgment from Cycle 3? Investigate any mismatch.	Hands-on review

Week 2 — Harness + Methodology Write-Up (6h)

Day	Time	Content	
Sun–Mon	3h	Finalize the evaluation harness: answer relevance, latency, and faithfulness/hallucination check all running together on one command.	
Tue	1.5h	Write a short methodology note: what's measured, how, and what the numbers actually mean (not just a results table with no interpretation).	
Wed	1h	Folded-in refresher (G-05): revisit Precision/Recall/F1 properly — your interview answer was vague ("closeness to the target"). This is cheap to close and will resurface in any evaluation conversation going forward.	Evaluation Metrics in Machine Learning — GeeksforGeeks + Precision, Recall & F1 Score Intuitively Explained
Thu	0.5h	Checkpoint prep.	

Deliverable

A lightweight evaluation harness added to the Cycle 3 RAG system, measuring at minimum: answer relevance, latency, and a basic faithfulness/hallucination check — plus a short methodology write-up, and a corrected, precise explanation of Precision/Recall/F1.

Checkpoint — pass criteria (mentor reviews methodology, not just resulting numbers)

- [] you can explain *why* each metric was chosen, not just report the number it produced
- [] Latency is broken down by pipeline stage, not reported as one opaque total
- [] your Precision/Recall/F1 explanation is now precise and distinguishes all three cleanly, unaided

Cycles 5–6 (Weeks 9–12): Applied to a Real Product Feature ★ scope-decision gate

Budget: 2 × ~12h · No new external sources — this is Cycles 1–4 applied to real, delivered work

Decision to make at the Week 8 1:1

Confirm with Bassem/mentor whether your 3D/VR skill becomes a genuine secondary contribution (e.g. a property-visualization feature for Amlak/Okade, floated live in your interview) or stays out of scope for this internship track. Either way, AI Engineering remains the primary track — this decision is about a possible *addition*, not a role split with graphic design (which the interviewers also floated and which this plan explicitly does not pursue, per Open Questions #4).

Week 9–10 — Integration (12h)

Focus	Content
Feature selection	Pick a real feature on your assigned project that genuinely needs LLM/RAG capability — not a retrofit for its own sake.
Build	Integrate Cycles 1–3 (LLM API call, retrieval, generation) into the real feature, reusing your own pipeline code rather than starting fresh.
Evaluate	Wire in the Cycle 4 evaluation harness against this real feature's actual queries, not the practice dataset.
Coaching checkpoint	Mid-cycle 1:1: name explicitly that you've gone from "haven't researched RAG" to shipping a real RAG-integrated feature in 8 weeks — a concrete, measurable G-06 (research efficiency) data point worth stating plainly.

Week 11–12 — Delivery + Retro (12h)

Focus	Content
Finish & harden	Handle edge cases the practice cycles didn't cover (empty retrieval results, malformed queries, real production traffic patterns).
Ship	Get the feature through normal PR review as real, delivered work — not a learning-repo exercise.
Retro	Short self-written retro: which of Cycles 1–4 transferred cleanly to real work, and which

Focus	Content
	needed real adjustment once outside the practice dataset.

Deliverable

A real, shipped feature on your assigned project using LLM API integration and RAG, evaluated with your own harness — normal PR review, not a separate learning checkpoint.

Checkpoint — pass criteria

- [] Feature passes normal Handbook DoD review without the reviewer needing to ask for missing error handling or evaluation
- [] you can speak to at least one thing that worked differently in production than in the practice cycles, and how you adapted
- [] **G-06 named explicitly:** the 12-week arc from self-admitted RAG novice to shipped feature is stated plainly at the monthly 1:1 as evidence of both research efficiency and coding speed improving — not left implicit

Standing Rhythm (applies to every cycle)

- **Daily:** learning block logged in Orbit as a `learning` task. If blocked >30 min on setup/tooling → ask, per Handbook culture. Blockers on learning count as blockers.
- **Weekly (Thursday, 10 min):** mentor confirms on-track/behind — no content review, just trajectory.
- **Cycle end:** checkpoint (criteria above) + 15-min mini-retro: what was over/under-budgeted? Feed that into the next cycle's plan.
- **Monthly 1:1 (Handbook §13):** competency outcomes from completed cycles reviewed against this document; G-06 (research efficiency, coding speed) revisited with concrete instances, not impressions.

Open Questions (carried from v1, still blocking precision)

9. **RockAI's actual RAG/LLM tooling stack** — which LLM provider, which vector DB, and whether a framework (LangChain/LlamaIndex) or a custom approach is preferred. This materially reshapes Cycles 1–3; the plan defaults to Anthropic + LangChain as the most likely fit but flags this explicitly rather than assuming it.
10. **your 3D/VR skill for real-estate visualization** — the interviewers floated this live. Worth a real decision at the Week 8 1:1 on whether it becomes a genuine secondary contribution rather than staying a footnote.
11. **Project assignment** — same open item as the rest of the cohort; most relevant to Cycles 3–6, which all target real, assigned-project work.
12. **Track confirmation** — the interviewer partly considered you for a graphic-design role. This plan assumes AI Engineering is the primary track and any 3D work is a secondary contribution, not a role split; worth confirming explicitly rather than leaving it ambiguous.
13. **Transcript gap** — ~2,900 characters covering your live answers on fine-tuning vs. RAG and LLM evaluation were cut off. If the full section becomes available, it may sharpen or shift the read on G-01/G-03.